

Transformer Optimization Closes Most of the Gap

TL;DR

The original fairness-first study seemed to show a large LSTM advantage over a small causal Transformer on Chinese name generation. After reviewer feedback, I reran the Transformer with a more appropriate optimization recipe: AdamW, linear warmup, cosine decay, lower dropout, and two peak learning rates. I also ran a light LSTM tuning symmetry check so the updated comparison would not become “tuned Transformer versus untuned LSTM.”

The result changes the story substantially.

At full data and temperature 0.8:

Model / recipe	Mean test PPL	Std
LSTM, flat Adam	77.9209	0.0342
LSTM tuned, AdamW + dropout 0.2	78.0511	0.0243
Transformer, flat Adam	86.7124	0.4884
Transformer tuned, peak 5e-4	79.0579	0.1839
Transformer tuned, peak 1e-3	79.0286	0.1572

The updated conclusion is:

Transformer optimization explains most of the original gap. With warmup and cosine decay, the small Transformer nearly catches the LSTM. A light LSTM tuning check did not improve the LSTM baseline, so the remaining small LSTM edge looks less like a one-sided tuning artifact. Still, the old large-gap architecture claim was too strong.

Why the Story Changed

The previous report correctly documented that both models used the same flat Adam recipe. That looked fair at first glance, but it created an optimization confounder.

Transformers are sensitive to early optimizer dynamics. Training a Transformer with flat Adam and no warmup can produce unstable early updates and poor convergence. The original Transformer plateau was real under that recipe, but it was not enough evidence for a strong architectural conclusion.

The tuned rerun keeps the Transformer architecture fixed and changes only the training recipe:

- optimizer: AdamW

- weight decay: 0.01
- scheduler: 10% linear warmup, then cosine decay
- dropout: 0.1
- peak learning rates: 5e-4 and 1e-3
- seeds: 13, 37, 73
- fraction: 100%
- temperature: 0.8

To avoid introducing a new asymmetry, I then reran the same LSTM architecture with a light tuned recipe:

- optimizer: AdamW
- weight decay: 0.01
- scheduler: flat
- dropout: 0.2
- learning rate: 1e-3
- seeds: 13, 37, 73
- fraction: 100%
- temperature: 0.8

This did not improve the LSTM. Its mean test perplexity moved from 77.9209 to 78.0511. I therefore keep the original flat-Adam LSTM as the stronger LSTM reference, while reporting the tuned check as a guard against tuning imbalance.

New Main Figure

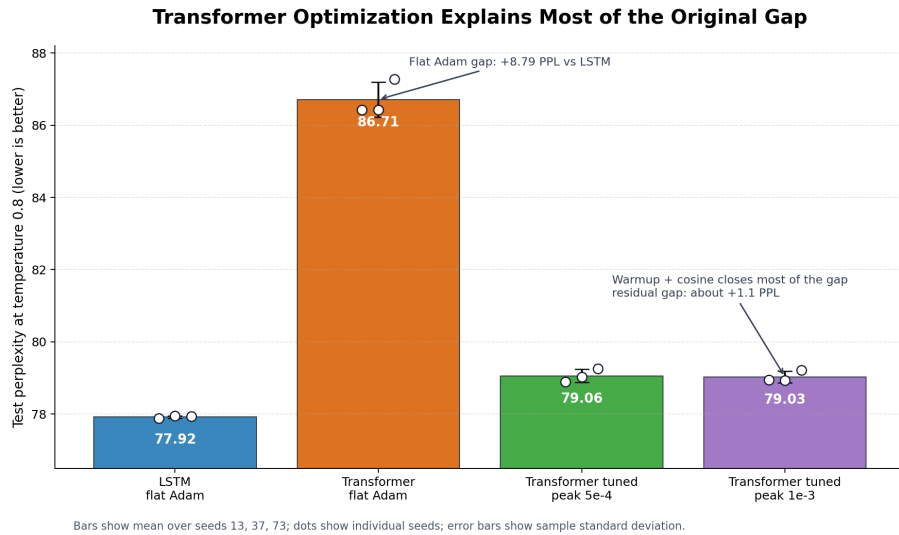


Figure 1: Transformer optimization comparison

This is now the main result figure. It compares:

- LSTM flat Adam
- LSTM tuned symmetry check, reported in the table above
- Transformer flat Adam
- Transformer tuned peak $5\text{e-}4$
- Transformer tuned peak $1\text{e-}3$

The figure makes the key point visually: the flat-Adam Transformer is a weak baseline. Warmup plus cosine closes most of the original gap. The LSTM tuned check is mainly a methodological control: it did not produce a stronger LSTM baseline.

Data and Vocabulary

The source data comes from the China Biographical Database (CBDB).

- cleaned full names: 584,442
- split: 80% train / 10% validation / 10% test
- split seed: 20260419
- training seeds: 13, 37, 73

Tokenization is a naive character-level mapping over individual Unicode characters observed in training names, plus <SOS>, <EOS>, <PAD>, and <UNK>.

There is no BPE, unigram tokenization, subword regularization, or compound-character vocabulary. The shared full-data vocabulary is about 8.4k tokens.

Model Configuration

The LSTM-small-matched model:

- parameters: 5,469,500
- embedding dimension: 160
- hidden size: 320
- layers: 2
- directionality: strictly forward, not bidirectional
- dropout in original run: 0.4
- dropout in tuned symmetry check: 0.2

The Transformer-small model:

- parameters: 5,355,708
- embedding dimension: 256
- layers: 2
- attention heads: 8
- feed-forward dimension: 512
- positional encoding: learned absolute positional embeddings
- max sequence length: 10
- dropout in original flat-Adam run: 0.4
- dropout in tuned runs: 0.1

The tuned Transformer keeps the same architecture as `transformer_small`.

The tuned LSTM keeps the same architecture as `lstm_small_matched`.

Metrics

Teacher-forced metrics:

- `cross_entropy`: mean cross-entropy over non-PAD target tokens.
- `perplexity`: $\exp(\text{cross_entropy})$, using natural-log base `e`.

Generation metrics:

- `unique_name_ratio`: unique generated full names divided by total generated names.
- `Distinct-1`: unique generated characters divided by total generated characters.
- `Distinct-2`: unique generated character bigrams divided by total generated character bigrams.
- `novelty_ratio`: generated names not appearing exactly in the training set divided by total samples.
- `train_overlap_ratio`: generated names appearing exactly in the training set divided by total samples.
- `consecutive_duplicate_ratio`: share of names with adjacent repeated characters.
- `any_duplicate_ratio`: share of names with any repeated character.
- `invalid_output_ratio`: share of empty, malformed, or otherwise invalid outputs.

Generation metrics use pure temperature-scaled categorical sampling. No top-k or top-p sampling is used in the reported metrics.

Revised Interpretation

The original result:

LSTM test PPL 77.9209, Transformer flat-Adam test PPL 86.7124.

This was a large gap, but it was mostly a training-recipe gap.

The tuned result:

LSTM test PPL 77.9209, tuned Transformer test PPL about 79.03.

The LSTM symmetry check:

LSTM flat-Adam test PPL 77.9209, LSTM AdamW/dropout-0.2
test PPL 78.0511.

This is much closer than the original Transformer result, and the LSTM tuning check does not reveal a hidden stronger LSTM baseline. The LSTM still has a

small edge of about 1.1 perplexity points over the best tuned Transformer run, but that is a very different claim from the old 8.79 point gap.

The defensible public claim is now:

Proper Transformer optimization closes most of the original gap. In the tested recipes, the LSTM still retains a small residual advantage on full-data test perplexity, and a light LSTM tuning symmetry check did not improve the LSTM baseline.

What Remains Open

The next study round should test:

- tuned Transformer at 10% and 30% data fractions
- additional LSTM dropout values, especially 0.1
- paired per-sequence log-likelihood tests for the residual gap
- weight tying between input embeddings and output projection
- top-k and top-p decoding
- larger matched pairs
- lightweight blinded human evaluation

Until those are done, this should not be published as a strong architecture verdict.

Conclusion

This project became more interesting after the reviewer feedback.

The old story was simpler:

LSTM beats Transformer.

The new story is better:

The naive Transformer baseline was under-optimized. Once tuned, the Transformer nearly catches up. A light LSTM tuning check does not improve the LSTM baseline, so the remaining small gap is worth reporting, but cautiously. The main lesson is that optimization and architecture cannot be separated.

That is a more mature result, and a much stronger portfolio story.